HTB Academy — Course Summary and Lab Notes

FortiLogger · Metasploit · Windows Privilege Context · NTLM Password Hashes

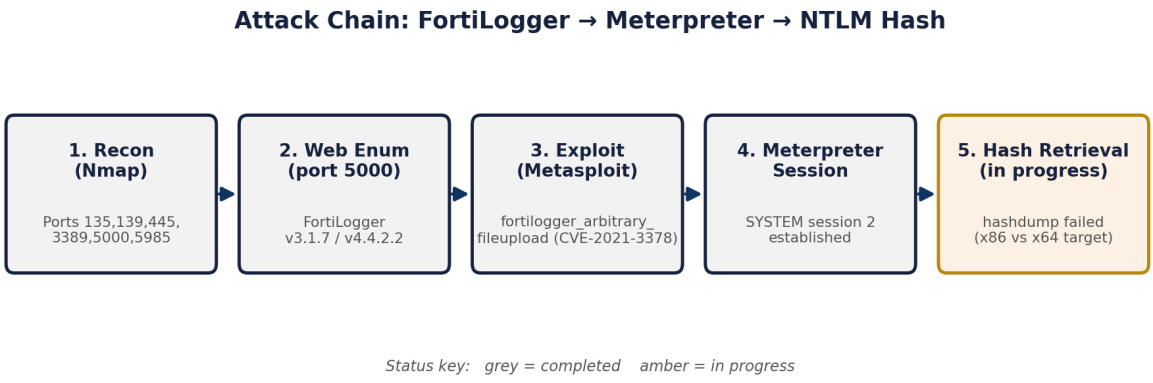
---

Based on our conversation, you're working through an HTB Academy practical lab involving FortiLogger, Metasploit, Windows privilege context, and NTLM password hashes.

I can summarise the material we've covered so far, including the commands, what they do, what you've successfully completed, and where you're stuck. I don't have the full text of the Academy module itself in this conversation, so this is a summary of our lab work rather than a claim to cover every lesson in the course.

1. The main learning objectives

- Identify exposed services on a Windows target.
- Inspect a web application and identify its version.
- Find and run a relevant Metasploit exploit.
- Establish a remote Meterpreter session.
- Check the account and privileges under which a session runs.
- Understand Windows NTLM password hashes and troubleshoot hash retrieval.



2. Reconnaissance and target discovery

The target used in your lab is **10.129.203.65**.

Your Nmap scan identified these services:

Port	Service	What it tells you
135	RPC	Windows remote procedure calls
139	NetBIOS	Legacy Windows networking
445	SMB	Windows file sharing and related services

3389	RDP	Remote Desktop
5000	IIS web application	The FortiLogger application
5985	WinRM	Windows remote management over HTTP

The target was identified as Windows Server 2019, build 17763, with hostname `WIN-51BJ97BCIPV`.

Key lesson: Port scans help you build an initial picture of a target, but an open port alone does not prove a vulnerability.

3. Web application enumeration

You inspected the website running on port 5000 and found:

- Application name: FortiLogger
- Version information in the page source: `3.1.7`
- Login endpoint: `/Login/Login`
- Username field: `txtusername`
- Password field: `txtpassword`

Metasploit's vulnerability check subsequently identified the installed FortiLogger version as **4.4.2.2**.

The differing version strings are worth remembering: a version string in a web page or JavaScript asset is not necessarily the authoritative version of the installed server application.

4. Finding the exploit in Metasploit

The module you found was:

```
exploit/windows/http/fortilogger_arbitrary_fileupload
```

The module targets an arbitrary-file-upload vulnerability associated with **CVE-2021-3378** in vulnerable FortiLogger versions.

The module's automatic check reported:

```
[+] The target is vulnerable. FortiLogger version 4.4.2.2
```

It then reported that the payload was uploaded and executed successfully.

Key lesson: Metasploit modules typically provide options, a vulnerability check where supported, and an exploit action. A positive check is useful evidence, but it is not the same as proving that a remote session has been established.

5. Establishing a Meterpreter session

Your exploit succeeded and opened a Meterpreter session.

The important output was:

```
Meterpreter session 2 opened
10.10.14.74:4444 -> 10.129.203.65:49696
```

Here:

- `10.10.14.74` is your VPN-side callback address.
- `4444` is the listener port you configured.
- `10.129.203.65` is the target.
- `Meterpreter session 2` is the active remote session.

You successfully established access to the target. You also reran the exploit, which opened a second session. You can list active sessions from the main Metasploit console using:

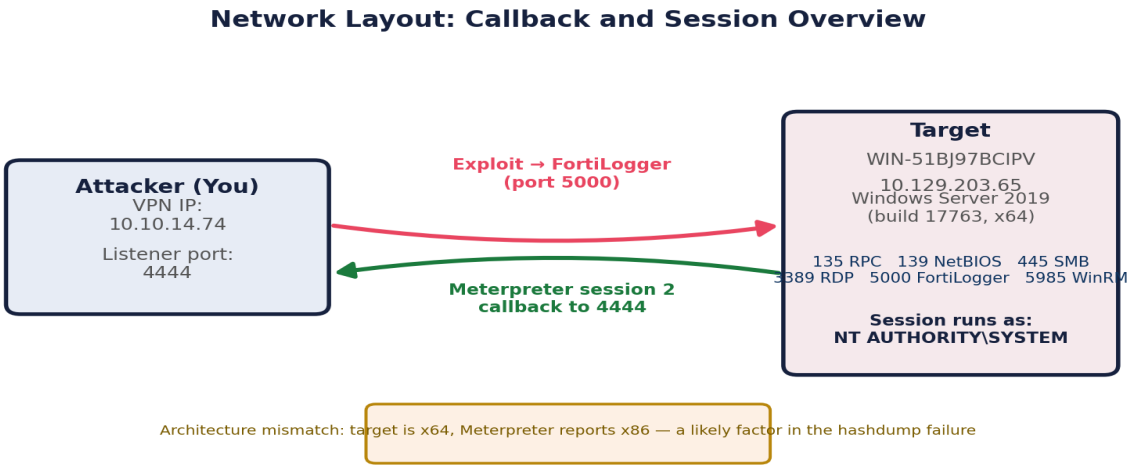
```
sessions -l
```

To return to an existing session:

```
sessions -i 2
```

To leave a Meterpreter session running in the background without closing it, use:

```
background
```



6. Understanding Windows identity and privileges

You ran the following commands inside Meterpreter.

getuid

Output:

```
Server username: NT AUTHORITY\SYSTEM
```

This means your session was running under the Windows SYSTEM account, a highly privileged built-in account.

This answers your first Academy question: the username you obtained a shell with is `NT AUTHORITY\SYSTEM`.

getprivs

Your output showed enabled privileges including:

- `SeDebugPrivilege` — permits certain debugging operations on processes.
- `SeBackupPrivilege` — permits certain backup-related access that can bypass normal file permissions.
- `SeRestorePrivilege` — permits certain restoration-related operations.
- `SeImpersonatePrivilege` — permits certain token impersonation operations.

These are Windows security privileges. Their presence is useful information when assessing the capabilities of a session, but they do not guarantee that every command or API operation will succeed.

sysinfo

Your output identified:

Property	Value
Computer	WIN-51BJ97BCIPV
Operating system	Windows Server 2019
Architecture	x64
Domain	WORKGROUP
Meterpreter architecture	x86/windows

The target operating system is 64-bit, while your Meterpreter session is 32-bit. This mismatch is a possible troubleshooting clue, although the output alone does not establish that it caused your hash-dumping error.

7. NTLM password hashes

Your second Academy question asks you to retrieve the NTLM password hash belonging to `htb-student`.

Windows stores local account information in the Security Account Manager (SAM) database. A Windows password hash is not the plaintext password; it is a cryptographic representation used by Windows authentication mechanisms.

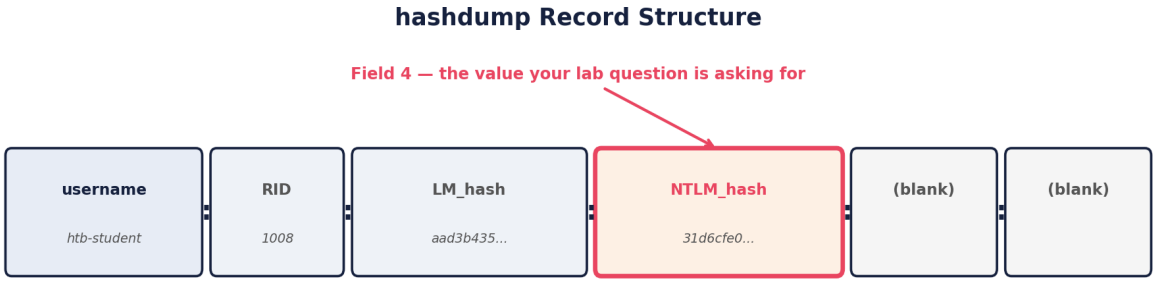
A typical hash-dump record has this structure:

```
username:RID:LM_hash:NTLM_hash:::
```

The fields mean:

- `username` — the account name.
- `RID` — the account's relative identifier.
- `LM_hash` — the legacy LAN Manager hash field.
- `NTLM_hash` — the NT hash field requested by your question.

If a lab asks specifically for the `htb-student` NTLM hash, the requested value is the **fourth colon-separated field** in that account's record, not the RID or the LM hash.



Why your hashdump command failed

You ran:

```
hashdump
```

And received:

```
[-] priv_passwd_get_sam_hashes: Operation failed: Incorrect function.
```

This means the attempted hash-retrieval operation failed. It does not mean that your Meterpreter session was lost or that you lack all privileges.

Your current findings are:

- You have a working Meterpreter session.
- Your session runs as `NT AUTHORITY\SYSTEM`.
- Several powerful Windows privileges are enabled.
- The target is x64, while Meterpreter reports x86.
- The requested hash has not yet been retrieved.

The error alone does not establish the precise cause. The session architecture, the retrieval implementation, and access to the relevant Windows security data are all things to investigate.

8. Your current position in the lab

Task 1 — Get a shell on the target

✔ Completed

You obtained a Meterpreter session running as `NT AUTHORITY\SYSTEM`.

Task 2 — Retrieve the htb-student NTLM hash

■ In progress

The `hashdump` command returned an error, so the actual hash is still unknown.

9. Quick revision sheet

Command / Concept	Purpose
nmap	Discover ports and services
Web page source	Inspect application clues and version strings
search (Metasploit)	Find relevant modules
show options	Review module configuration
check	Test for vulnerability where supported
run	Execute the selected module
sessions -l	List active sessions
sessions -i 2	Interact with session 2
background	Leave a session open in the background
getuid	Identify the session's Windows account
getprivs	Inspect enabled process privileges
sysinfo	Inspect operating system and architecture
hashdump	Attempt to retrieve local account hashes

10. What I need to make the summary cover the entire course

If you want a genuinely complete summary of the whole HTB Academy module — including every section, explanation, exercise, and key takeaway — send me the course or module title, or upload its course notes/material. I'll organise it into a study guide with definitions, commands, examples, and revision questions, keeping the course's terminology and separating material that hasn't been provided from what we know from your lab.

For now, the immediate goal is to resolve the failed hash retrieval while preserving your working session.

11. The process explained, step by step

This section pulls everything above into a single narrative, so you can see how each step led to the next and why it mattered.

Why recon came first

An Nmap scan against `10.129.203.65` was the starting point because you can't attack what you haven't found. The scan returned six open ports, and of those, port 5000 stood out as unusual — it's not a standard Windows service port, which is what pointed you toward a custom web application (FortiLogger) rather than a built-in Windows feature. The other ports (135, 139, 445, 3389, 5985) are all normal for a Windows host and, on their own, didn't give you a way in.

Why the version mismatch mattered

The web page reported FortiLogger 3.1.7, but Metasploit's own check against the live application reported 4.4.2.2. This is a common situation in real assessments: a version string baked into a web page or JavaScript file is often stale or deliberately vague, while a tool that actually probes the running service gets a more accurate answer. You correctly trusted Metasploit's live check over the page source when choosing which exploit to run.

Why the exploit worked

`fortilogger_arbitrary_fileupload` targets CVE-2021-3378, a flaw that lets an attacker upload a file to the server without proper validation. Because the upload path allows arbitrary files, the exploit uploads a malicious payload disguised as a legitimate file, then gets the server to execute it. Metasploit's `check` command confirmed the target was vulnerable before you ran it, which is why you could be confident the exploit had a real chance of success rather than just hoping it would work.

Why the session runs as SYSTEM

The payload executed with the same privileges as the FortiLogger service itself. Many Windows applications, especially ones installed as services, run under the SYSTEM account by default rather than a limited user account. That's a configuration weakness in its own right: if the application is compromised, the attacker inherits SYSTEM-level control immediately, with no separate privilege escalation step required. That's exactly what `getuid` confirmed.

Why hashdump is failing

`hashdump` needs to read protected areas of the Windows registry (the SAM and SYSTEM hives) using low-level Windows API calls. Those calls are sensitive to the architecture of the process making them. Your Meterpreter session is a 32-bit (x86) process running on a 64-bit (x64) target. A 32-bit process has a different, restricted view of the registry on a 64-bit system (part of what Windows calls WOW64 redirection), and some privileged operations — hash extraction among them — simply don't work correctly across that boundary. The "Incorrect function" error is consistent with this: Meterpreter is asking Windows to do something that the current process context can't carry out, not that the session has lost its privileges.

The standard fix is to migrate the session into a native 64-bit process (for example with `migrate` to a 64-bit process ID, or by using a 64-bit payload from the start) and then retry `hashdump` from inside that process.

12. Answers to your Academy questions

Question 1 — What is the username you obtained a shell with?

Answer: NT AUTHORITY\SYSTEM

This came directly from running `getuid` inside the Meterpreter session, which reported `Server username: NT AUTHORITY\SYSTEM`. SYSTEM is the highest-privileged built-in Windows account — higher than a local administrator — and the exploit landed you there because the vulnerable FortiLogger service itself runs under that account.

Question 2 — What is the NTLM hash of the user htb-student?

Answer: not yet retrieved — hashdump is failing

This one is still open. Running `hashdump` returned `Operation failed: Incorrect function`, most likely because the Meterpreter session is a 32-bit process on a 64-bit target (see Section 11 above). Once the session is migrated to a 64-bit process and `hashdump` succeeds, the answer will be the fourth colon-separated field in the `htb-student` line of the output — the value labelled `NTLM_hash` in the Section 7 diagram — not the RID and not the LM hash field before it.